

- AxonRouter Feedback System Documentation
 - Overview
 - Part 1: Mask Submit Redrawing Component (Primary Feedback System)
 - Overview
 - 1.1 Component Architecture
 - Frontend Implementation
 - 1.2 Technical Implementation Details
 - Canvas Initialization Flow
 - Drawing Tools Configuration
 - Canvas State Management
 - 1.3 Data Flow Architecture
 - Mask-to-Canvas Conversion
 - Canvas Data Structure
 - 1.4 Azure ML Integration
 - File Upload System
 - Azure ML Data Asset Creation
 - 1.5 User Experience Flow
 - 1.6 Current Implementation Status
 - Implemented Features:
 - [x]Azure ML Integration:
 - Part 2: Sidebar Checkbox Feedback Component (Secondary System)
 - Overview
 - 2.1 Component Architecture
 - Frontend Implementation
 - 2.2 Feedback Categories
 - Prediction Quality Indicators:
 - Image Quality Issues:
 - 2.3 Data Processing & Storage
 - CSV File Structure
 - Submission Process
 - 2.4 Integration Limitations
 - Current Status:
 - Conditional Display Logic:
 - 2.5 System Integration Status
 - Working Components:
 - 2.6 Current Implementation Gaps
 - Sidebar Feedback:

- [Canvas Corrections:](#)
- [Integration Bridge:](#)
- [Summary](#)
 - [Primary System \(Canvas - Fully Functional\):](#)
 - [Secondary System \(Sidebar - Partially Functional\):](#)
 - [Key Technical Achievement:](#)
 - [Current Limitation:](#)

AxonRouter Feedback System Documentation

Overview

The AxonRouter application implements a comprehensive two-part feedback system that captures both detailed mask corrections and structured categorical feedback to improve model performance through user interactions. The system is designed to work with Azure ML endpoints for root segmentation prediction and provides multiple pathways for user feedback.

Part 1: Mask Submit Redrawing Component (Primary Feedback System)

Overview

The Mask Submit page is the core feedback mechanism allowing users to interactively correct AI predictions using a drawable canvas. This component captures detailed mask corrections and provides functionality to upload corrected datasets to Azure ML datastores for model retraining.

1.1 Component Architecture

Frontend Implementation

Location: `render_mask_submit_page()` function (lines 2197-2600+)

Core Components:

1. Canvas Configuration System
2. Interactive Drawing Controls
3. Streamlit Drawable Canvas Integration
4. State Management
5. Azure ML Data Asset Upload

1.2 Technical Implementation Details

Canvas Initialization Flow

```
# Location: render_mask_submit_page()
def render_mask_submit_page():
    # 1. Validate prediction data exists
    pad_img_np = st.session_state.get("pad_img_np", None)
    pred_mask = st.session_state.get("pred_mask", None)
    uploaded_filename = st.session_state.get("uploaded_filename", None)

    # 2. Load original prediction as initial drawing
    canvas_init_key = f"canvas_init_{uploaded_filename}"
    if canvas_init_key not in st.session_state and pred_mask is not None:
        update_canvas_with_prediction_mask(opacity=opacity)
        st.session_state[canvas_init_key] = True

    # 3. Configure drawing tools
    drawing_controls = create_inline_drawing_controls()

    # 4. Initialize canvas with prediction mask as background
    canvas_result = st_canvas(
        background_image=background_image, # Original grayscale image
        initial_drawing=initial_drawing,   # Prediction mask converted to
Fabric.js
        drawing_mode=canvas_config["drawing_mode"],
        # ... additional configuration
    )
```

Drawing Tools Configuration

Function: `create_inline_drawing_controls()`

Available Tools:

- **Freedraw:** Continuous line drawing for organic corrections

- **Line:** Straight line tool for precise boundaries
- **Eraser:** Remove incorrect predictions or drawn areas

Tool Parameters:

```
drawing_controls = {  
    "drawing_mode": "freedraw",      # Tool type  
    "stroke_width": 3,               # Brush thickness (1-20)  
    "opacity": 0.7,                  # Stroke transparency (0.1-1.0)  
    "realtime_update": True,         # Live canvas updates  
    "display_toolbar": True          # Show canvas toolbar  
}
```

Canvas State Management

Session State Variables:

```
# Per-image canvas state preservation  
canvas_state_key = f"canvas_state_{uploaded_filename}"  
st.session_state[canvas_state_key] = canvas_result.json_data  
  
# Initial drawing state (prediction mask)  
st.session_state.canvas_initial_drawing = fabric_js_json  
  
# Tool state preservation  
st.session_state.previous_drawing_mode = current_mode  
st.session_state.previous_opacity = current_opacity
```

1.3 Data Flow Architecture

Mask-to-Canvas Conversion

Function: `convert_mask_to_canvas_json()`

```
# Convert numpy prediction mask to Fabric.js JSON format  
def convert_mask_to_canvas_json(pred_mask, canvas_width=600, canvas_height=600,  
    opacity=0.7):  
    # Process: Numpy Array → Contour Detection → Fabric.js Paths  
    return {  
        "version": "5.2.4",  
        "objects": [  
            {  
                "type": "path",  
                "path": "M 10 10 L 50 50 Z", # SVG path data
```

```

        "stroke": f"rgba(0, 255, 0, {opacity})",
        "strokeWidth": 3,
        "fill": "transparent"
    }
]
}

```

Canvas Data Structure

Output Format: Fabric.js JSON containing user corrections

```

{
  "version": "5.2.4",
  "objects": [
    {
      "type": "path",
      "path": "M 123 45 L 150 78 Q 200 100 250 120",
      "stroke": "rgba(0, 255, 0, 0.7)",
      "strokeWidth": 3,
      "fill": "transparent",
      "left": 123,
      "top": 45,
      "width": 127,
      "height": 75
    }
  ]
}

```

1.4 Azure ML Integration

File Upload System

Function: `upload_files_to_azure_datastore()`

The system now provides a proof-of-concept Azure ML integration where users can:

1. **Upload Original Images:** Multiple JPG/JPEG/PNG files
2. **Upload Mask Files:** Multiple TIF mask files
3. **Submit Canvas Corrections:** Fabric.js JSON data from drawings

```

# Submission flow in render_mask_submit_page()
if st.button("Submit Corrections"):
    # Step 1: Validate uploads and canvas data
    if not uploaded_ct_images and not uploaded_ct_masks and
canvas_result.json_data.get("objects", []) == []:

```

```

        st.warning("Please upload files or draw corrections before submitting.")
        return

# Step 2: Test Azure CLI connection
cli_working, cli_message = test_azure_cli_connection_with_workspace()
if not cli_working:
    st.error(f"❌ Azure CLI Issue: {cli_message}")
    return

# Step 3: Upload to Azure ML datastore
data_asset_name, saved_files, upload_result = upload_files_to_azure_datastore(
    uploaded_ct_images, uploaded_ct_masks, canvas_result.json_data
)

```

Azure ML Data Asset Creation

The upload process creates structured datasets in Azure ML:

```

# Data organization structure
saved_files = {
    'train_images': [...], # 80% of uploaded images
    'train_masks': [...], # 80% of uploaded masks
    'val_images': [...], # 20% of uploaded images
    'val_masks': [...], # 20% of uploaded masks
}

```

1.5 User Experience Flow

1. User completes prediction on Prediction & Analysis page
↓
2. Prediction results stored in session state (pad_img_np, pred_mask)
↓
3. User navigates to Mask Submit page
↓
4. System validates prediction data exists
↓
5. Original prediction mask converted to Fabric.js format
↓
6. Canvas initialized with:
 - Background: Original grayscale image
 - Initial Drawing: Prediction mask as green overlay
 ↓
7. User selects drawing tool (freedraw/line/eraser)
↓
8. User draws corrections directly on canvas
↓
9. Canvas state automatically saved to session state
↓

10. User uploads additional images/masks (optional)
↓
11. User clicks "Submit Corrections"
↓
12. System validates Azure CLI connection
↓
13. Files uploaded to Azure ML datastore as new data asset
↓
14. Success confirmation with asset details displayed

1.6 Current Implementation Status

☑ Implemented Features:

- Interactive canvas with prediction mask overlay
- Multiple drawing tools (freedraw, line, eraser)
- Real-time tool parameter adjustment
- Canvas state preservation across tool and config changes
- Original prediction display for reference
- Reset to original prediction functionality
- **Azure ML datastore upload integration**
- **Proof-of-concept dataset submission workflow**

[x] Azure ML Integration:

- Azure CLI connection validation
- Data asset creation in Azure ML workspace
- Structured file organization (train/val splits)
- Canvas JSON data inclusion in submissions
- Progress tracking and error handling

Part 2: Sidebar Checkbox Feedback Component (Secondary System)

Overview

The sidebar feedback form provides structured, categorical feedback about prediction quality. This complements the detailed mask corrections with quantitative metrics and saves data locally for analysis.

2.1 Component Architecture

Frontend Implementation

Location: `main()` function, sidebar section (lines 2867-2940)

Form Structure:

```
if pad_img_np is not None and tips: # Only show if prediction exists with tips
    with st.sidebar.form("feedback"):
        st.markdown("***Help us improve the model***")
        st.info("Select all issues that apply:")

        # Feedback categories
        fb_correct = st.checkbox("Tips are correct", key="tips_correct")
        fb_outofbounds = st.checkbox("Tips out of bounds", key="tips_oob")
        fb_rootoverlay = st.checkbox("Root overlay interferes", key="root_overlay")
        fb_thinroot = st.checkbox("Thin roots interfere", key="thin_root")
        fb_missing_tips = st.checkbox("Missing tips", key="missing_tips")
        fb_wrong_tips = st.checkbox("Wrong tips", key="wrong_tips")
        fb_obs_root = st.checkbox("Obscured roots", key="obscured_roots")

        submitted = st.form_submit_button("Submit Feedback")
```

2.2 Feedback Categories

Prediction Quality Indicators:

1. **Tips are correct** - Positive feedback for accurate detection
2. **Missing tips** - AI failed to detect visible root tips
3. **Wrong tips** - AI detected objects that are not root tips
4. **Tips out of bounds** - Detected tips are outside image boundaries

Image Quality Issues:

5. **Root overlay interferes** - Overlapping roots confuse detection
6. **Thin roots interfere** - Very thin roots cause false positives

7. Obscured roots - Partially hidden roots affect accuracy

2.3 Data Processing & Storage

CSV File Structure

Location: `{app_directory}/feedback.csv`

```
file_name,tips_correct,tips_oob,root_overlay,thin_root,missing_tips,wrong_tips,obscured_roots
example_image.jpg,True,False,False,True,False,True,False
another_image.png,False,False,True,False,True,False,True
```

Submission Process

```
if submitted:
    with st.spinner("Saving feedback..."):
        try:
            feedback_file = os.path.join(os.path.dirname(__file__), "feedback.csv")
            file_exists = os.path.isfile(feedback_file)

            with open(feedback_file, "a", newline="") as csvfile:
                writer = csv.writer(csvfile)
                if not file_exists:
                    # Write headers for new file
                    writer.writerow([
                        "file_name", "tips_correct", "tips_oob", "root_overlay",
                        "thin_root", "missing_tips", "wrong_tips", "obscured_roots"
                    ])

                # Write feedback data
                file_name = st.session_state.get("uploaded_filename", "")
                feedback_data = [
                    file_name, fb_correct, fb_outofbounds, fb_rootoverlay,
                    fb_thinroot, fb_missing_tips, fb_wrong_tips, fb_obs_root
                ]
                writer.writerow(feedback_data)

            st.session_state.feedback_submitted = True

        except Exception as e:
            st.error(f"Error saving feedback: {str(e)}")
```

2.4 Integration Limitations

Current Status:

☑ **Functional Local Storage:** CSV file creation and data persistence

☑ **Form Validation:** Proper error handling and user feedback

☐ **Azure ML Integration:** Not implemented - feedback only saved locally

☐ **Backend API Connection:** No REST API endpoint for feedback submission

Conditional Display Logic:

The feedback form only appears when:

- `pad_img_np is not None` - A prediction has been completed
- `tips` - Root tips have been detected (current implementation requirement)

Note: In the current "no tips" version, the feedback form may not appear since the condition requires `tips` to exist.

2.5 System Integration Status

Working Components:

1. Canvas Correction System [x]

- Interactive drawing interface
- Azure ML datastore upload
- Data asset creation workflow

2. Structured Feedback System !

- Form functionality works
- Local CSV storage works
- **Limited by tips detection requirement**

3. Azure ML Integration !

- ☑ Implemented
- Structured feedback upload: ☐ Not implemented

2.6 Current Implementation Gaps

Sidebar Feedback:

☒ Form collection and validation

☒ Local CSV storage

☐ Azure ML integration

☐ Backend API connectivity

! Display Condition Issue: Form requires **tips** to exist, which may not be available in current "no tips" implementation

Canvas Corrections:

☒ Interactive drawing and editing

☒ Session state preservation

☒ Fabric.js data format

☒ Azure ML datastore upload

☒ Persistent storage via Azure ML

Integration Bridge:

☐ Unified submission workflow

☐ Data linking between components

☐ Azure ML pipeline triggers

Summary

The current feedback system implementation provides:

Primary System (Canvas - Fully Functional):

- Complete interactive mask correction interface
- Full Azure ML integration with datastore uploads
- Proof-of-concept workflow for dataset submission
- Session state management and drawing persistence

Secondary System (Sidebar - Partially Functional):

- Working form interface with categorical feedback options
- Local CSV storage and data persistence
- **Limited availability due to tips detection requirement**
- No Azure ML integration for structured feedback data

Key Technical Achievement:

The mask submit system successfully demonstrates end-to-end Azure ML integration, providing a functional pathway for users to submit corrected training data directly to Azure ML datastores for potential model retraining workflows.

Current Limitation:

The sidebar feedback system's dependency on **tips** detection may prevent it from displaying in the current implementation that focuses on mask-only predictions without tip detection.